syndicate

Encode XML and generate Atom feeds, RSS feeds and sitemaps

Version 0.1.0 · Typst 0.15.0 or later
https://github.com/bfeitknecht/syndicate

syndicate encodes Typst values to XML and generates Atom 1.0 feeds, RSS 2.0 feeds and XML sitemaps for websites and blogs built with Typst. It is written in pure Typst, with no plugins. Licensed under Apache-2.0; a fork of exemel by Jeremy Gao.

Contents

1 Introduction 3
2 Getting started 4
2.1 One source of truth 4
2.2 Writing the files 4
2.3 Text, dates and time zones 5
3 XML 6
3.1 Encoding 6
3.2 Decoding 6
4 Feeds 8
4.1 Atom 8
4.2 RSS 9
5 Sitemaps 10
6 Extending documents 11
7 Migrating from exemel 12
8 Reference 13
8.1 xml 13
8.2 atom 17
8.3 rss 24
8.4 sitemap 31
8.5 Dates 34

1 Introduction

Typst can export HTML, which makes it a capable static site generator. A website rarely consists of pages alone, though: readers subscribe to **feeds**, and search engines crawl **sitemaps**. Both are XML documents, and Typst can read XML but not write it.

syndicate fills that gap with four modules:

Module	Contents
`xml`	`encode` and `decode` for arbitrary XML, plus node helpers
`atom`	Atom 1.0 feeds (RFC 4287): `feed`, `entry`, `author`, `tag-uri`
`rss`	RSS 2.0 feeds: `feed`, `item`, `enclosure`
`sitemap`	XML sitemaps: `urlset`, `url`, and sitemap indexes with `index`

The date formatters `rfc3339` and `rfc2822` are exported as well. Import what you need:

```
#import "@preview/syndicate:0.1.0": xml, atom, rss, sitemap
```

Every document function returns the finished XML as a string. Importing `xml` shadows Typst's built-in `xml()` reader in your file; use `xml.decode`, which wraps the reader, or `std.xml`.

2 Getting started

2.1 One source of truth

Keep the metadata of your posts in one place and map it to every format:

```
#import "@preview/syndicate:0.1.0": atom, rss, sitemap

#let site = "https://example.com/"
#let posts = (
  (
    title: "Hello, world",
    slug: "hello-world",
    date: datetime(year: 2026, month: 7, day: 1),
    summary: [My _first_ post],
  ),
  (title: "Second post", slug: "second", date: "2026-07-14T09:30", summary: "More things"),
)
#let url(p) = site + "blog/" + p.slug + "/"

#let feed = atom.feed(
  title: "My Blog",
  link: site,
  self-link: site + "feed.xml",
  authors: "Basil",
  entries: posts.map(p => atom.entry(
    p.title,
    link: url(p),
    published: p.date,
    summary: p.summary,
  )),
)

#let rss-feed = rss.feed(
  title: "My Blog",
  link: site,
  description: "Things I write",
  self-link: site + "rss.xml",
  items: posts.map(p => rss.item(
    p.title,
    link: url(p),
    published: p.date,
    description: p.summary,
  )),
)

#let map = sitemap.urlset(site, posts.map(p => sitemap.url(url(p), lastmod: p.date)))
```

2.2 Writing the files

By default, a Typst document produces a single output. There are two ways to get the XML out.

With bundle export. Typst 0.15's experimental bundle target writes any number of files, next to your HTML pages if you have any:

```
#asset("feed.xml", feed)
#asset("rss.xml", rss-feed)
#asset("sitemap.xml", map)
```

```
typst compile --features bundle --format bundle blog.typ site/
```

With `typst eval`. Expose the strings with `metadata` and extract them on the command line. In Typst 0.15, `eval` prints JSON, so unwrap the string with `jq`:

```
#metadata(feed) <feed>
```

```
typst eval --in blog.typ 'query(<feed>).first().value' | jq -j . > feed.xml
```

2.3 Text, dates and time zones

Text fields accept strings, as well as Typst content made of plain markup: text, spaces, smart quotes, `*strong*`, `_emph_`, `raw` , links, underline and the like. Feeds can only hold plain text, so markup is stripped, and `[My _first_ post]` becomes `My first post`. Content without a faithful plain-text form, such as equations, footnotes or images, is an error that names the field. Nothing is dropped silently. When in doubt, pass a string.

Dates are `datetime` values or ISO 8601 strings without a time zone: `"2026-07-01"`, `"2026-07-01T14:30"` or `"2026-07-01T14:30:00"`. Strings are handy for front matter loaded from YAML or JSON. Dates without a time get `00:00:00`, except in sitemaps, where they stay date-only.

Time zones are set once per document with `tz`: `"Z"` (the default), `"+01:00"` or `"-0530"`. Typst's `datetime` has no time zone of its own, so every date in a document is interpreted in `tz`.

```
rfc3339("2026-07-01", tz: "+02:00")
```

↓

```
2026-07-01T00:00:00+02:00
```

3 XML

3.1 Encoding

`xml.encode` serializes a tree of nodes. Elements are dictionaries with a `tag` and optional `attrs` and `children`, the same shape Typst's `xml()` reader produces. Text is a string, a number or simple content, and is escaped automatically.

```
xml.encode(
  (
    tag: "bookstore",
    children: (
      xml.comment(" inventory "),
      (
        tag: "book",
        attrs: (id: "bk101", category: "programming", discontinued: none),
        children: (
          (tag: "title", attrs: ("xml:lang": "en"), children: ("XML & Typst",)),
          (tag: "price", children: (44.95,)),
        ),
      ),
    ),
  ),
  pretty: true,
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <!-- inventory -->
  <book id="bk101" category="programming">
    <title xml:lang="en">XML &amp; Typst</title>
    <price>44.95</price>
  </book>
</bookstore>
```

- **Attributes** keep their order. `none` omits an attribute; numbers and booleans are converted to text.
- **Children** may contain `none`, which is skipped, and nested arrays, which are flattened, so `if` expressions and `.map(..)` can be used inline.
- **`pretty`** only indents elements that contain nothing but other elements. Whitespace is never added next to text, so text content is preserved exactly.
- **Namespace prefixes** are part of the name: `(tag: "atom:link")`. The `namespace` key holds a namespace URI, see Section 3.2.
- **Invalid input is an error:** invalid names, characters XML doesn't allow, `--` in comments, several root elements, misspelled keys.

Use `xml.comment`, `xml.cdata` and `xml.pi` for the other node types, and `xml.raw` to insert pre-serialized markup verbatim.

3.2 Decoding

`xml.decode` parses XML into the same structure, using Typst's built-in reader. It accepts a string, bytes, or a `path(..)` to a file. Use `path` rather than a plain file name, so the file is found relative to your document rather than to this package.

```gleam
let doc = xml.decode("<list><item>a</item><item>b</item></list>").first()
let items = doc.children.filter(c => type(c) == dictionary)
xml.encode(
  (..doc, children: items + ((tag: "item", children: ("c",)),)),
  declaration: false,
)
```

↓

```
<list><item>a</item><item>b</item><item>c</item></list>
```

Decoded elements carry a `namespace` key with their namespace URI, which `xml.encode` turns back into `xmlns` declarations, so namespaced documents round-trip:

```gleam
xml.encode(
  xml.decode("<r xmlns='urn:a' xmlns:b='urn:b'><b:x/><y/></r>"),
  declaration: false,
)
```

↓

```
<r xmlns="urn:a"><x xmlns="urn:b"/><y/></r>
```

Decoding inherits two limitations of the built-in reader:

- Comments and processing instructions are dropped. The reader leaves empty placeholder elements in their place, which `xml.decode` removes; `xml.encode` rejects them in plain `std.xml` output.
- Prefixes are lost: `<media:thumbnail>` comes back as `thumbnail` with the media namespace URI, and `xml:lang` comes back as `lang`.

4 Feeds

Atom and RSS are both widely supported by feed readers. Atom is the better specified of the two; RSS is older and more common in podcasting. Offering both costs one more call.

4.1 Atom

```
atom.feed(
  title: "My Blog",
  link: "https://example.com/",
  self-link: "https://example.com/feed.xml",
  id: atom.tag-uri("example.com", "2026-01-01", "blog"),
  authors: atom.author("Basil", email: "basil@example.com"),
  tz: "+01:00",
  entries: atom.entry(
    "Hello, world",
    link: "https://example.com/blog/hello/",
    id: atom.tag-uri("example.com", "2026-07-01", "blog/hello"),
    published: "2026-07-01",
    updated: "2026-07-03T14:30",
    summary: [My _first_ post],
    content: "<p>Welcome to my <em>new</em> blog.</p>",
    categories: ("meta", (term: "typst", label: "Typst")),
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <title>My Blog</title>
  <link rel="alternate" type="text/html" href="https://example.com/"/>
  <link rel="self" type="application/atom+xml" href="https://example.com/feed.xml"/>
  <id>tag:example.com,2026-01-01:blog</id>
  <updated>2026-07-03T14:30:00+01:00</updated>
  <author>
    <name>Basil</name>
    <email>basil@example.com</email>
  </author>
  <entry>
    <title>Hello, world</title>
    <link rel="alternate" type="text/html" href="https://example.com/blog/hello/"/>
    <id>tag:example.com,2026-07-01:blog/hello</id>
    <updated>2026-07-03T14:30:00+01:00</updated>
    <published>2026-07-01T00:00:00+01:00</published>
    <category term="meta"/>
    <category term="typst" label="Typst"/>
    <summary>My first post</summary>
    <content type="html">&lt;p&gt;Welcome to my &lt;em&gt;new&lt;/em&gt; blog.&lt;/p&gt;</
content>
  </entry>
</feed>
```

Every feed and entry needs a permanent, unique `id`. It defaults to the `link`, which changes when you move your site; `atom.tag-uri` builds identifiers that don't. Every entry needs an author, set on the feed or on each entry. A string `content` is HTML source, escaped as Atom requires; Typst content is converted to plain text.

4.2 RSS

```
rss.feed(
  title: "My Podcast",
  link: "https://example.com/",
  description: "Conversations about typesetting",
  self-link: "https://example.com/rss.xml",
  language: "en-us",
  items: rss.item(
    "Episode 1",
    link: "https://example.com/podcast/1/",
    published: "2026-07-08T06:00",
    author: (name: "Basil", email: "basil@example.com"),
    enclosure: rss.enclosure(
      "https://example.com/podcast/1.mp3",
      length: 24986239,
      type: "audio/mpeg",
    ),
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0" xmlns:atom="http://www.w3.org/2005/Atom">
  <channel>
    <title>My Podcast</title>
    <link>https://example.com/</link>
    <description>Conversations about typesetting</description>
    <atom:link href="https://example.com/rss.xml" rel="self" type="application/rss+xml"/>
    <language>en-us</language>
    <lastBuildDate>Wed, 08 Jul 2026 06:00:00 +0000</lastBuildDate>
    <item>
      <title>Episode 1</title>
      <link>https://example.com/podcast/1/</link>
      <author>basil@example.com (Basil)</author>
      <enclosure url="https://example.com/podcast/1.mp3" length="24986239" type="audio/mpeg"/>
      <guid isPermaLink="true">https://example.com/podcast/1/</guid>
      <pubDate>Wed, 08 Jul 2026 06:00:00 +0000</pubDate>
    </item>
  </channel>
</rss>
```

RSS's `<author>` requires an email address, so authors given only by name are written as `<dc:creator>`. The `atom`, `content` and `dc` namespaces are declared automatically when they're used.

5 Sitemaps

A sitemap lists the pages of a website for search engines. Pages are `sitemap.url(..)` entries or plain URL strings, passed as arguments or arrays.

```
sitemap.urlset(
  sitemap.url("https://example.com/", changefreq: "daily", priority: 1),
  sitemap.url(
    "https://example.com/about/",
    lastmod: "2026-05-01",
    alternates: (de: "https://example.com/de/about/"),
  ),
  "https://example.com/imprint/",
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" xmlns:xhtml="http://www.w3.org/
1999/xhtml">
  <url>
    <loc>https://example.com/</loc>
    <changefreq>daily</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://example.com/about/</loc>
    <lastmod>2026-05-01</lastmod>
    <xhtml:link rel="alternate" hreflang="de" href="https://example.com/de/about/"/>
  </url>
  <url>
    <loc>https://example.com/imprint/</loc>
  </url>
</urlset>
```

A sitemap holds at most 50 000 URLs. Split larger sites and list the parts with `sitemap.index`.

6 Extending documents

Every document function accepts `tz`, `stylesheet` and `pretty`:

- **`stylesheet`** adds an `<?xml-stylesheet?>` instruction, so browsers show a styled page instead of raw XML. Both XSL and CSS are supported.
- **`pretty`** indents the output and is on by default.

`atom.feed`, `rss.feed` and `sitemap.urlset` also accept `namespaces`, extra `(prefix: uri)` declarations on the root element. The feeds and their entries, as well as `sitemap.url`, accept `extra`: XML nodes appended to the element. Together, they support extensions such as Media RSS:

```
rss.feed(
  title: "Gallery",
  link: "https://example.com/",
  description: "Pictures",
  namespaces: (media: "http://search.yahoo.com/mrss/"),
  items: rss.item(
    "Sunset",
    link: "https://example.com/sunset/",
    extra: (tag: "media:content", attrs: (url: "https://example.com/sunset.jpg")),
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0" xmlns:media="http://search.yahoo.com/mrss/">
  <channel>
    <title>Gallery</title>
    <link>https://example.com/</link>
    <description>Pictures</description>
    <item>
      <title>Sunset</title>
      <link>https://example.com/sunset/</link>
      <guid isPermaLink="true">https://example.com/sunset/</guid>
      <media:content url="https://example.com/sunset.jpg"/>
    </item>
  </channel>
</rss>
```

7 Migrating from exemel

syndicate started as a fork of exemel and redesigns its API:

exemel	**syndicate**
`to-xml(value, pretty: ..)`	`xml.encode(value, pretty: ..)`
`namespace: "atom"` (a prefix)	`tag: "atom:link"`; `namespace` holds a URI
attributes sorted alphabetically	attributes keep their order
`atom-encode(site-link: .., posts: ..)`	`atom.feed(link: .., entries: ..)`
`feed-link`, guessed as `feed.xml`	`self-link`, omitted unless given
`atom-post(..)`	`atom.entry(..)`; `id` defaults to `link`
`atom-author(..)`	`atom.author(..)`, or a string
`atom-site-id(authority: .., time: .., identifier: ..)`	`atom.tag-uri(authority, date, specific)`
`tz` on every post	`tz` once per document
Typst 0.13 or later, WASM plugin	Typst 0.15 or later, pure Typst

8 Reference

8.1 xml

- escape()
- element()
- comment()
- cdata()
- pi()
- raw()
- encode()
- decode()

escape

Escapes text for use as XML character data: &, < and > become entity references.

```
xml.escape("Fish & <Chips>")
```

↓

```
Fish &amp; &lt;Chips&gt;
```

Parameters

```
escape(text: str content) -> str
```

text `str` or `content`

The text.

element

Builds an element node. Equivalent to writing the dictionary `(tag: tag, attrs: attrs, children: children)` yourself.

```
xml.element("a", attrs: (href: "/"), "Home")
```

↓

```
(tag: "a", attrs: (href: "/"), children: ("Home",))
```

Parameters

```
element(
  tag: str,
  attrs: dictionary,
  ..children: str content dictionary array none
) -> dictionary
```

tag `str`

The element name, optionally with a namespace prefix like `"atom:link"`.

attrs `dictionary`

The attributes. Attributes set to `none` are omitted.

Default: `(:)`

..children `str` or `content` or `dictionary` or `array` or `none`

The child nodes.

comment

A comment node, written as `<!--body-->`.

```
xml.encode(
  (tag: "p", children: (xml.comment(" draft "),)),
  declaration: false,
)
```

↓

```
<p><!-- draft --></p>
```

Parameters

`comment(body: str) -> dictionary`

body `str`

The comment text. It must not contain `--` or end with `-`.

cdata

A CDATA section, written as `<![CDATA[body]]>`. Occurrences of `]]>` are split across two sections, so any text is safe.

```
xml.encode(
  (tag: "script", children: (xml.cdata("if (a < b) go()"),)),
  declaration: false,
)
```

↓

```
<script><![CDATA[if (a < b) go()]]></script>
```

Parameters

```
cdata(body: str) -> dictionary
```

body `str`

The text.

pi

A processing instruction, written as `<?target data?>`.

```
xml.encode(
  (xml.pi("xml-stylesheet", "href=\"style.xsl\""), (tag: "doc")),
  declaration: false,
)
```

↓

```
<?xml-stylesheet href="style.xsl"?><doc/>
```

Parameters

```
pi(
  target: str,
  data: str none
) -> dictionary
```

target `str`

The target, e.g. `"xml-stylesheet"`.

data `str` or `none`

The instruction's data. It must not contain `?>`.

raw

Pre-serialized markup, written verbatim. You are responsible for it being well-formed.

```
xml.encode(
  (tag: "p", children: (xml.raw("<b>bold</b> &amp; raw"),)),
  declaration: false,
)
```

↓

```
<p><b>bold</b> &amp; raw</p>
```

Parameters

```
raw(body: str) -> dictionary
```

body `str`

The markup.

encode

Serializes a tree of nodes to an XML document.

Elements are dictionaries `(tag: .., attrs: .., children: ..)`, where `attrs` and `children` are optional. Text is given as strings, numbers or simple content and is escaped automatically. Children set to `none` are skipped and nested arrays are flattened, so `if` and `.map(..)` can be used inline.

```
xml.encode(
  (
    tag: "book",
    attrs: (id: "bk101", lang: none),
    children: (
      (tag: "title", children: ("XML & Typst",)),
      (tag: "price", children: (44.95,)),
    ),
  ),
  pretty: true,
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<book id="bk101">
  <title>XML &amp;amp; Typst</title>
  <price>44.95</price>
</book>
```

Parameters

```
encode(
  root: dictionary array,
  pretty: bool,
  indent: int str,
  declaration: bool str
) -> str
```

root `dictionary` or `array`

The root element, or an array of top-level nodes: exactly one element, optionally surrounded by comments and processing instructions.

pretty `bool`

Whether to indent elements that contain only other elements. Text content is never altered.

Default: `false`

indent `int` or `str`

The indentation per level when `pretty` is set: a number of spaces, or a string such as `"\t"`.

Default: `2`

declaration `bool` or `str`

`true` for `<?xml version="1.0" encoding="UTF-8"?>`, `false` for no declaration, or a custom declaration.

Default: `true`

decode

Parses XML into the node structure used by `xml.encode`, using Typst's built-in reader.

Elements are returned as `(namespace: .., tag: .., attrs: .., children: ..)`, where `namespace` is the element's namespace URI, which `xml.encode` turns back into `xmlns` declarations. The reader drops comments, processing instructions and namespace prefixes.

```
xml.decode("<p>Hello <b>world</b></p>")
```

↓

```
(
  (
    namespace: none,
    tag: "p",
    attrs: (:),
    children: (
      "Hello ",
      (
        namespace: none,
        tag: "b",
        attrs: (:),
        children: ("world",),
      ),
    ),
  ),
)
```

Parameters

`decode(source: str bytes path) -> array`

source `str` or `bytes` or `path`

The XML as a string or bytes, or a `path(..)` to a file. Use `path` rather than a plain file name, so that the file is found relative to your document instead of this package.

8.2 atom

- tag-uri()
- author()
- entry()

- feed()

tag-uri

Builds a `tag:` URI (RFC 4151): a permanent, unique identifier for a feed or entry that survives changes to your URLs.

```
atom.tag-uri("example.com", "2026-07-01", "blog/hello-world")
```

↓

```
tag:example.com,2026-07-01:blog/hello-world
```

Parameters

```
tag-uri(
  authority: str,
  date: datetime str,
  specific: str
) -> str
```

authority `str`

A domain name or email address you controlled on `date`.

date `datetime` or `str`

A date on which you controlled `authority`, typically when the feed or entry was created.

specific `str`

Anything that is unique for this authority and date.

author

Describes a person, for the `authors` and `contributors` of feeds and entries. Wherever a person is expected, a plain string is accepted as well and used as the name.

Parameters

```
author(
  name: str content,
  email: str none,
  uri: str none
) -> dictionary
```

name `str` or `content`

The person's name.

email `str` or `none`

An email address.

Default: `none`

uri `str` or `none`

A web page.

Default: `none`

entry

Describes an entry, for `atom.feed(entries: ..)`.

```
atom.entry(
  "Hello, world",
  link: "https://example.com/blog/hello/",
  published: "2026-07-01",
  summary: [My _first_ post],
)
```

Parameters

```
entry(
  title: str content,
  id: str auto,
  link: str none,
  updated: datetime str auto,
  published: datetime str none,
  summary: str content none,
  content: str content none,
  content-type: str auto,
  authors: array,
  contributors: array,
  categories: array,
  rights: str content none,
  extra: array
) -> dictionary
```

title `str` or `content`

The title.

id `str` or `auto`

A permanent, unique identifier. Defaults to `link`; consider `atom.tag-uri` for identifiers that survive URL changes.

Default: `auto`

link `str` or `none`

The URL of the entry's web page. Required unless `content` is given.

Default: `none`

updated `datetime` or `str` or `auto`

When the entry last changed significantly. Defaults to `published`.

Default: `auto`

published `datetime` or `str` or `none`

When the entry was first published.

Default: `none`

summary `str` or `content` or `none`

A short plain-text summary.

Default: `none`

content `str` or `content` or `none`

The full content. A string is HTML source, escaped as Atom requires; content is converted to plain text.

Default: `none`

content-type `str` or `auto`

The content's `type`: `"html"`, `"text"` or a MIME type. By default `"html"` for strings and `"text"` for content.

Default: `auto`

authors `array`

The authors: strings or `atom.author(..)`.

Default: `()`

contributors `array`

The contributors: strings or `atom.author(..)`.

Default: `()`

categories `array`

The categories: strings, or dictionaries `(term: .., scheme: .., label: ..)`.

Default: `()`

rights `str` or `content` or `none`

A copyright notice.

Default: `none`

extra `array`

Additional XML nodes appended to the entry.

Default: `()`

feed

Generates an Atom 1.0 feed.

```
atom.feed(
  title: "My Blog",
  link: "https://example.com/",
  authors: "Basil",
  entries: atom.entry(
    "Hello, world",
    link: "https://example.com/blog/hello/",
    published: "2026-07-01",
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <title>My Blog</title>
  <link rel="alternate" type="text/html" href="https://example.com/"/>
  <id>https://example.com/</id>
  <updated>2026-07-01T00:00:00Z</updated>
  <author>
    <name>Basil</name>
  </author>
  <entry>
    <title>Hello, world</title>
    <link rel="alternate" type="text/html" href="https://example.com/blog/hello/"/>
    <id>https://example.com/blog/hello/</id>
    <updated>2026-07-01T00:00:00Z</updated>
    <published>2026-07-01T00:00:00Z</published>
  </entry>
</feed>
```

Parameters

```
feed(
  title: str content,
  subtitle: str content none,
  link: str,
  self-link: str none,
  id: str auto,
  updated: datetime str auto,
  authors: array str,
  contributors: array,
  categories: array,
  rights: str content none,
  icon: str none,
  logo: str none,
  generator: str dictionary none,
  language: str none,
  entries: array,
  tz: str,
  stylesheet: str none,
  namespaces: dictionary,
  extra: array,
  pretty: bool
) -> str
```

title `str` or `content`

The feed's title. Required.

Default: `none`

subtitle `str` or `content` or `none`

A subtitle or description.

Default: `none`

link `str`

The URL of the website. Required.

Default: `none`

self-link `str` or `none`

The URL where this feed is published. Recommended.

Default: `none`

id `str` or `auto`

A permanent, unique identifier. Defaults to `link`.

Default: `auto`

updated `datetime` or `str` or `auto`

When the feed last changed. Defaults to the latest `updated` of the entries.

Default: `auto`

authors `array` or `str`

The authors: strings or `atom.author(..)`. Required unless every entry has its own.

Default: `()`

contributors `array`

The contributors: strings or `atom.author(..)`.

Default: `()`

categories `array`

The categories: strings, or dictionaries `(term: .., scheme: .., label: ..)`.

Default: `()`

rights `str` or `content` or `none`

A copyright notice.

Default: `none`

icon `str` or `none`

The URL of a small icon (square, like a favicon).

Default: `none`

logo `str` or `none`

The URL of a larger logo (2:1 aspect ratio).

Default: `none`

generator `str` or `dictionary` or `none`

The software that generated the feed: a name, or a dictionary `(name: .., uri: .., version: ..)`.

Default: `none`

language `str` or `none`

The language of the feed, e.g. `"en"`, written as `xml:lang`.

Default: `none`

entries `array`

The entries, created with `atom.entry(..)`.

Default: `()`

tz `str`

The time zone of all dates: `"Z"` for UTC, or an offset like `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet that browsers apply to the feed.

Default: `none`

namespaces `dictionary`

Additional namespace declarations `(prefix: uri)`, for elements in `extra`.

Default: `(:)`

extra `array`

Additional XML nodes appended to the feed.

Default: `()`

pretty `bool`

Whether to indent the output.

Default: `true`

8.3 rss

- enclosure()
- item()
- feed()

enclosure

Describes a media attachment, such as a podcast episode, for `rss.item(enclosure: ..)`. All three values are required.

```
rss.enclosure(
  "https://example.com/episode-1.mp3",
  length: 24986239,
  type: "audio/mpeg",
)
```

Parameters

```
enclosure(
  url: str,
  length: int,
  type: str
) -> dictionary
```

url `str`

The URL of the file.

length `int`

The file size in bytes.

Default: none

type `str`

The MIME type, e.g. `"audio/mpeg"`.

Default: none

item

Describes an item, for `rss.feed(items: ..)`.

```
rss.item(
  "Hello, world",
  link: "https://example.com/blog/hello/",
  published: "2026-07-01",
  description: [My _first_ post],
)
```

Parameters

```
item(
  title: str content none,
  link: str none,
  guid: str auto none,
  permalink: bool auto,
  published: datetime str none,
  description: str content none,
  content: str none,
  author: dictionary str none,
  categories: array,
  comments: str none,
  enclosure: dictionary none,
  extra: array
) -> dictionary
```

title `str` or `content` or `none`

The title. May be `none` if there is a `description`.

link `str` or `none`

The URL of the item's web page.

Default: `none`

guid `str` or `auto` or `none`

A unique identifier. Defaults to `link`.

Default: `auto`

permalink `bool` or `auto`

Whether `guid` is a URL that points to the item. By default `true` for `http(s)://` identifiers.

Default: `auto`

published `datetime` or `str` or `none`

When the item was published.

Default: `none`

description `str` or `content` or `none`

A summary or the full text. HTML source is allowed; it is escaped as required.

Default: `none`

content `str` or `none`

The full content as HTML source, written as `<content:encoded>`.

Default: `none`

author `dictionary` or `str` or `none`

The author: a dictionary `(name: .., email: ..)`, an `atom.author(..)`, or a string. Authors with an email are written as `<author>`, names alone as `<dc:creator>`.

Default: `none`

categories `array`

The categories: strings, or dictionaries `(name: .., domain: ..)`.

Default: `()`

comments `str` or `none`

The URL of the item's comments.

Default: `none`

enclosure `dictionary` or `none`

A media attachment, created with `rss.enclosure(..)`.

Default: `none`

extra `array`

Additional XML nodes appended to the item.

Default: `()`

feed

Generates an RSS 2.0 feed. The `atom`, `content` and `dc` namespaces are declared automatically when they are used.

```
rss.feed(
  title: "My Blog",
  link: "https://example.com/",
  description: "Things I write",
  items: rss.item(
    "Hello, world",
    link: "https://example.com/blog/hello/",
    published: "2026-07-01",
```

```
  ),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0">
  <channel>
    <title>My Blog</title>
    <link>https://example.com/</link>
    <description>Things I write</description>
    <lastBuildDate>Wed, 01 Jul 2026 00:00:00 +0000</lastBuildDate>
    <item>
      <title>Hello, world</title>
      <link>https://example.com/blog/hello/</link>
      <guid isPermaLink="true">https://example.com/blog/hello/</guid>
      <pubDate>Wed, 01 Jul 2026 00:00:00 +0000</pubDate>
    </item>
  </channel>
</rss>
```

Parameters

```
feed(
  title: str content,
  link: str,
  description: str content,
  self-link: str none,
  language: str none,
  copyright: str content none,
  managing-editor: str none,
  web-master: str none,
  published: datetime str none,
  updated: datetime str auto none,
  categories: array,
  generator: str none,
  docs: str none,
  ttl: int none,
  image: str dictionary none,
  items: array,
  tz: str,
  stylesheet: str none,
  namespaces: dictionary,
  extra: array,
  pretty: bool
) -> str
```

title `str` or `content`

The feed's title. Required.

Default: none

link `str`

The URL of the website. Required.

Default: none

description `str` or `content`

A description of the feed. Required.

Default: none

self-link `str` or `none`

The URL where this feed is published, written as `<atom:link rel="self">`. Recommended.

Default: none

language `str` or `none`

The language of the feed, e.g. `"en-us"`.

Default: none

copyright `str` or `content` or `none`

A copyright notice.

Default: none

managing-editor `str` or `none`

The email address of the editor.

Default: none

web-master `str` or `none`

The email address of the webmaster.

Default: none

published `datetime` or `str` or `none`

When the feed's content was published.

Default: none

updated `datetime` or `str` or `auto` or `none`

When the feed last changed (`<lastBuildDate>`). Defaults to the latest `published` of the items; `none` omits it.

Default: `auto`

categories `array`

The categories: strings, or dictionaries `(name: .., domain: ..)`.

Default: `()`

generator `str` or `none`

The software that generated the feed.

Default: `none`

docs `str` or `none`

The URL of the RSS specification.

Default: `none`

ttl `int` or `none`

How many minutes the feed may be cached.

Default: `none`

image `str` or `dictionary` or `none`

An image: a URL, or a dictionary `(url: .., title: .., link: .., width: .., height: ..)`. `title` and `link` default to those of the feed.

Default: `none`

items `array`

The items, created with `rss.item(..)`.

Default: `()`

tz `str`

The time zone of all dates: `"Z"` for UTC, or an offset like `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet that browsers apply to the feed.

Default: `none`

namespaces `dictionary`

Additional namespace declarations `(prefix: uri)`, for elements in `extra`.

Default: `(:)`

extra `array`

Additional XML nodes appended to the channel.

Default: `()`

pretty `bool`

Whether to indent the output.

Default: `true`

8.4 sitemap

- url()
- urlset()
- index()

url

Describes a page, for `sitemap.urlset(..)`, or a sitemap, for `sitemap.index(..)`.

```
sitemap.url(
  "https://example.com/about/",
  lastmod: "2026-05-01",
  alternates: (de: "https://example.com/de/about/"),
)
```

Parameters

```
url(
  loc: str,
  lastmod: datetime str none,
  changefreq: str none,
  priority: int float none,
  alternates: dictionary,
  extra: array
) -> dictionary
```

loc `str`

The page's canonical URL.

lastmod `datetime` or `str` or `none`

When the page last changed. Dates without a time are written as `YYYY-MM-DD`.

Default: `none`

changefreq `str` or `none`

How often the page changes: `"always"`, `"hourly"`, `"daily"`, `"weekly"`, `"monthly"`, `"yearly"` or `"never"`.

Default: `none`

priority `int` or `float` or `none`

The page's priority relative to your other pages, from `0` to `1`.

Default: `none`

alternates `dictionary`

Translations of the page as `(hreflang: url)`, e.g. `(de: "https://example.com/de/", "x-default": "https://example.com/")`.

Default: `(:)`

extra `array`

Additional XML nodes appended to the entry.

Default: `()`

urlset

Generates a sitemap listing the pages of a website. Sitemaps are limited to 50 000 URLs; split larger sites and list the parts with `sitemap.index`.

```
sitemap.urlset(
  "https://example.com/",
  sitemap.url("https://example.com/about/", lastmod: "2026-05-01"),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://example.com/</loc>
  </url>
  <url>
    <loc>https://example.com/about/</loc>
    <lastmod>2026-05-01</lastmod>
```

```
  </url>
</urlset>
```

Parameters

```
urlset(
  ..urls: dictionary str array,
  tz: str,
  stylesheet: str none,
  namespaces: dictionary,
  pretty: bool
) -> str
```

..urls `dictionary` or `str` or `array`

The pages: `sitemap.url(..)` or plain URL strings, as arguments or arrays.

tz `str`

The time zone of dates with a time: `"Z"` for UTC, or an offset like `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet that browsers apply to the sitemap.

Default: `none`

namespaces `dictionary`

Additional namespace declarations `(prefix: uri)`, for elements in `extra`.

Default: `(:)`

pretty `bool`

Whether to indent the output.

Default: `true`

index

Generates a sitemap index, which lists several sitemaps.

```
sitemap.index(
  "https://example.com/sitemap-pages.xml",
```

```
  sitemap.url("https://example.com/sitemap-posts.xml", lastmod: "2026-07-03"),
)
```

↓

```
<?xml version="1.0" encoding="UTF-8"?>
<sitemapindex xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <sitemap>
    <loc>https://example.com/sitemap-pages.xml</loc>
  </sitemap>
  <sitemap>
    <loc>https://example.com/sitemap-posts.xml</loc>
    <lastmod>2026-07-03</lastmod>
  </sitemap>
</sitemapindex>
```

Parameters

```
index(
  ..sitemaps: dictionary str array,
  tz: str,
  stylesheet: str none,
  pretty: bool
) -> str
```

..sitemaps `dictionary` or `str` or `array`

The sitemaps: URL strings, or `sitemap.url(loc, lastmod: ..)`.

tz `str`

The time zone of dates with a time: `"Z"` for UTC, or an offset like `"+01:00"`.

Default: `"Z"`

stylesheet `str` or `none`

The URL of an XSL or CSS stylesheet that browsers apply to the index.

Default: `none`

pretty `bool`

Whether to indent the output.

Default: `true`

8.5 Dates

- rfc3339()
- rfc2822()

rfc3339

Formats a date as an RFC 3339 timestamp, as used by Atom feeds and sitemaps. Dates without a time get `00:00:00`.

```
rfc3339("2026-07-01T14:30", tz: "+02:00")
```

↓

```
2026-07-01T14:30:00+02:00
```

Parameters

```
rfc3339(
  dt: datetime str,
  tz: str
) -> str
```

dt `datetime` or `str`

The date.

tz `str`

The time zone the date is in: `"Z"` for UTC, or an offset like `"+01:00"` or `"-0530"`.

Default: `"Z"`

rfc2822

Formats a date as an RFC 2822 timestamp, as used by RSS feeds. Dates without a time get `00:00:00`.

```
rfc2822("2026-07-01T14:30", tz: "+02:00")
```

↓

```
Wed, 01 Jul 2026 14:30:00 +0200
```

Parameters

```
rfc2822(
  dt: datetime str,
  tz: str
) -> str
```

dt `datetime` or `str`

The date.

tz `str`

The time zone the date is in: `"Z"` for UTC, or an offset like `"+01:00"` or `"-0530"`.

Default: `"Z"`